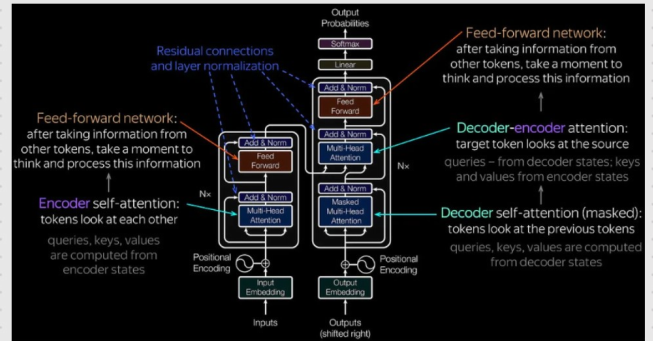


Introduction To Transformers

- 1) RNN/LSTM/GRU RNN
- 2) Encoder Decoder Architecture
- 3) ATTENTION MECHANISM
- 4) TRANSFORMERS

- ① Why Transformers?
- ② Architecture of Transformers?
- ③ SELF ATTENTION $\rightarrow Q, K, V$
- ④ Positional Encoding
- ⑤ Multi Head ATTENTION
- ⑥ Combining the working of Transformers

Architecture



Generative AI $\rightarrow$ LM, Multimodal

BERT, GPT $\leftarrow$



Open AI $\rightarrow$ Chat GPT

$\downarrow$
GPT-4o

① What And Why $\rightarrow$ Transformers

Transformers in natural language processing (NLP) are a type of deep learning model that use self-attention mechanisms to analyze and process natural language data. They are encoder-decoder models that can be used for many applications, including machine translation. $\Rightarrow$ Seq2Seq Task

Eg: Language Translation $\rightarrow$ Google Translation

English $\rightarrow$ French

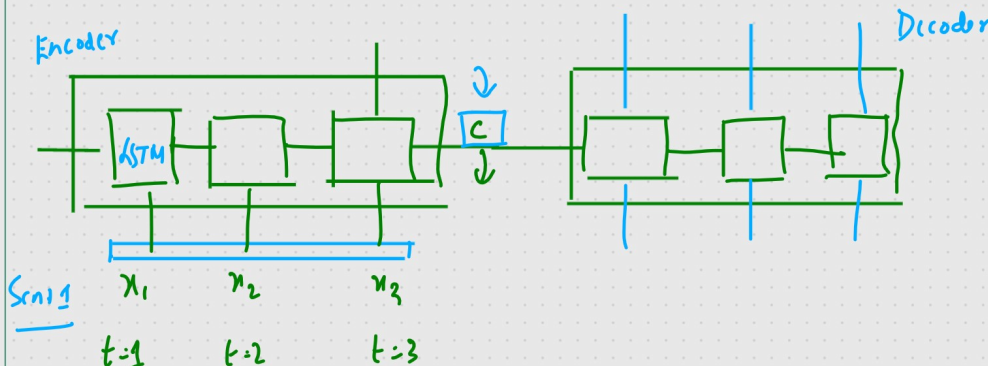
ip $\Rightarrow$ Many $\rightarrow$ o/p: Many. {Length of the sentence}.

Encoder - Decoder

Sentence Length $\uparrow\uparrow$

Bleau Score $\downarrow\downarrow$

Length Sentence $\uparrow\uparrow$.



3.1 DECODER: GENERAL DESCRIPTION

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i | y_1, \dots, y_{i-1}, x) = g(y_{i-1}, s_i, c_i), \quad (4)$$

where s_i is an RNN hidden state for time i , computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

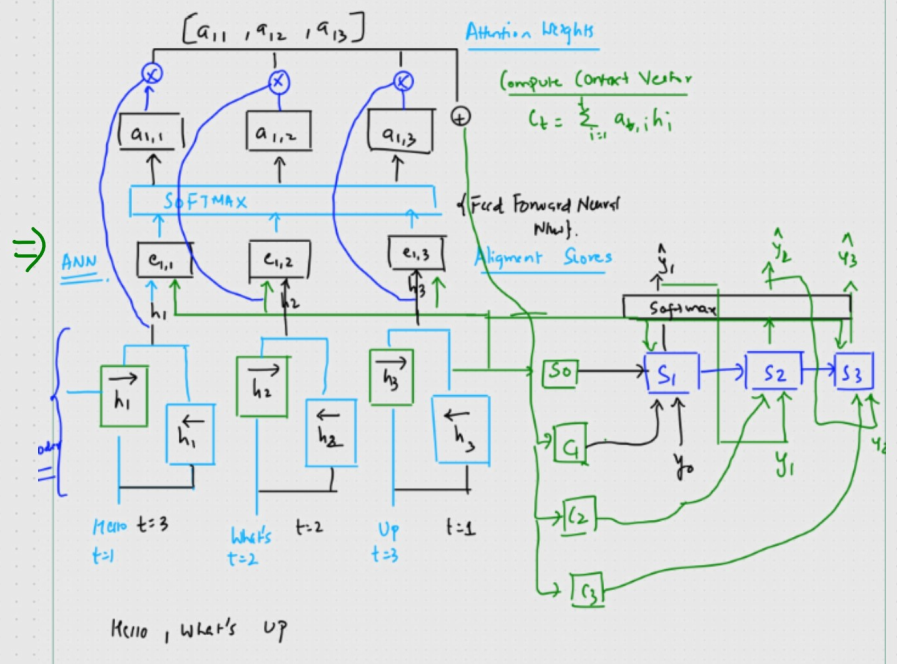
It should be noted that unlike the existing encoder-decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i .

The context vector c_i depends on a sequence of annotations $(h_1, \dots, h_T)$ to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i -th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_i :

$$c_i = \sum_{j=1}^T \alpha_{ij} h_j. \quad (5)$$

Figure 1: The graphical illustration of the proposed model trying to generate the i -th target word y_i given a source sentence $(x_1, x_2, \dots, x_T)$.



Additional Context $\rightarrow$ Decoder

Long Sentence. Accuracy $\uparrow\uparrow$.

Attention Mechanism

① Parallely We cannot send all the words in a sentence $\rightarrow$ Scalable

DATASET $\rightarrow$ Huge $\rightarrow$ Scalable With Respect To Training.

TRANSFORMERS $\neq$ LSTM RNN

$\hookrightarrow$ Self Attention Module $\leftarrow$ All the words will be parallelly sent to encoder.

$\downarrow$

Positional Encoding

Transformer $\uparrow\uparrow$ DATASET $\rightarrow$ Amazing SOTA $\leftarrow$ NLP

Transfer Learning $\rightarrow$ MultiModal Task $\rightarrow$ NLP + Image $\leftarrow$

Transformers $\div$ AI Space $\rightarrow$ SOTA Model $\rightarrow$

$\downarrow$ $\uparrow$ $\downarrow$

BERT GPT

$\uparrow$

Train Huge Data

$\rightarrow$ Transfer Learning $\rightarrow$ SOTA Models $\rightarrow$ DALL-E

LLM's

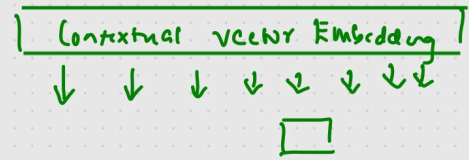
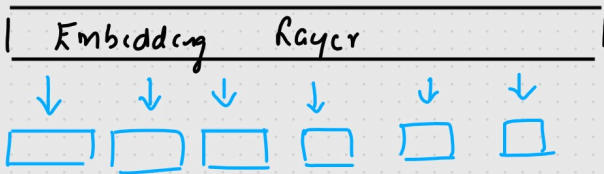
Generative AI

② Contextual Embedding → Self Attention

Contextual Vectors

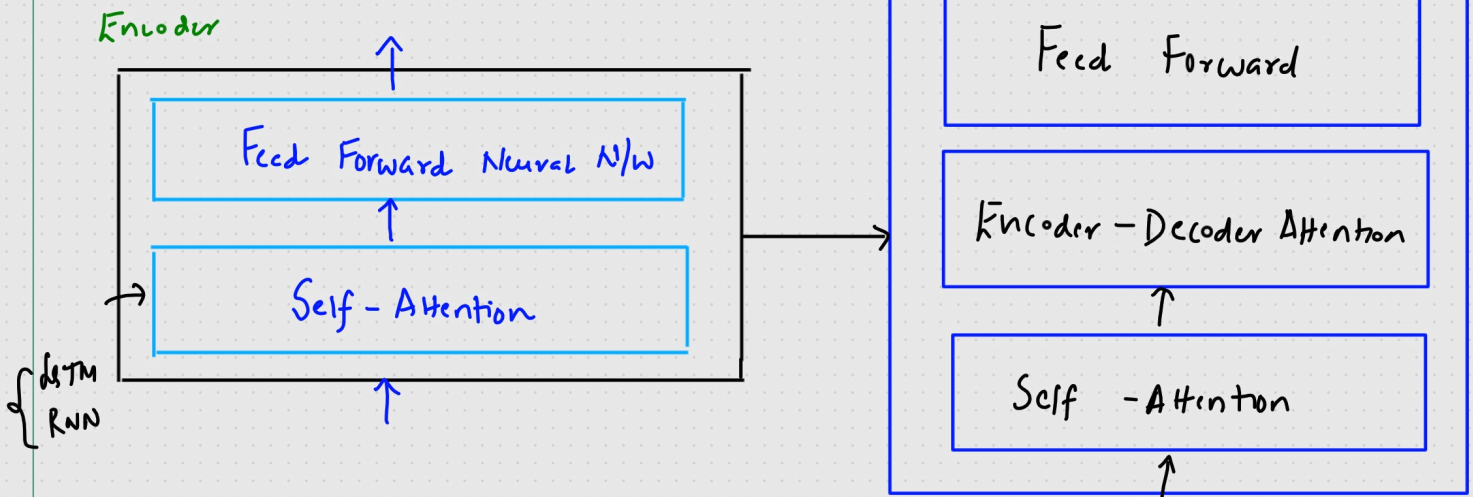
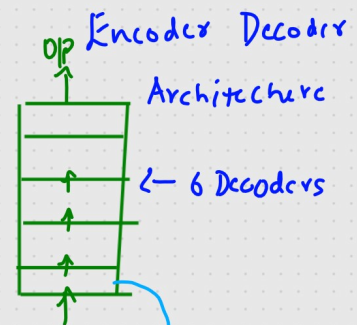
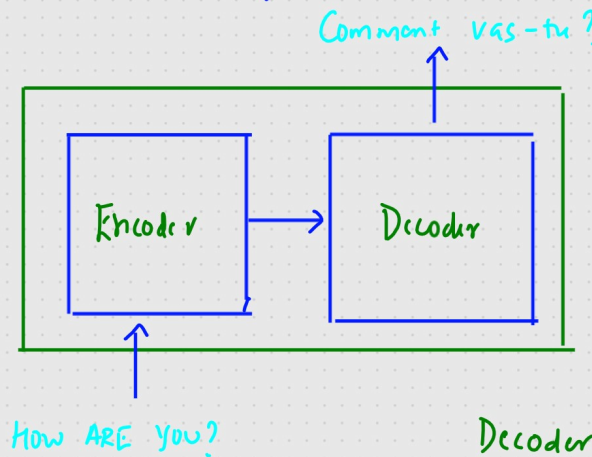
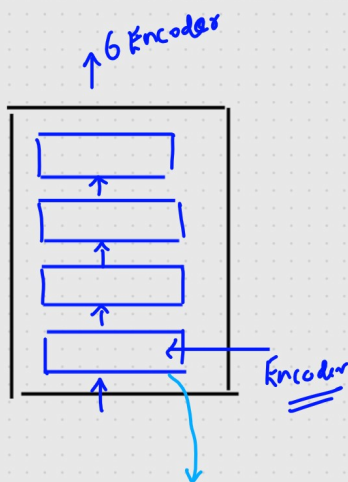
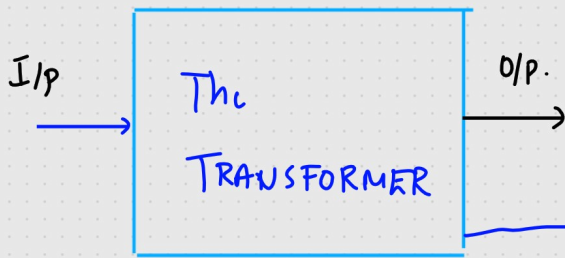
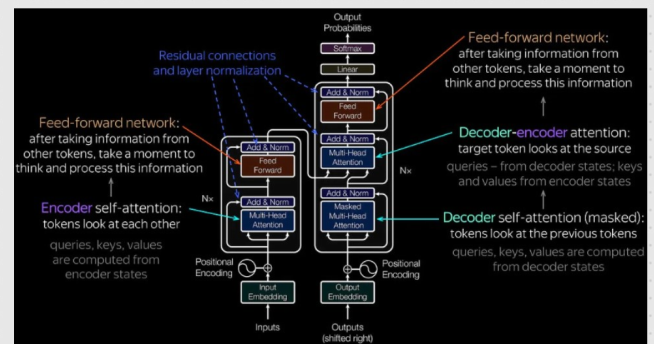
Eg: My name is KRISH And I play CRICKET

Word2vec

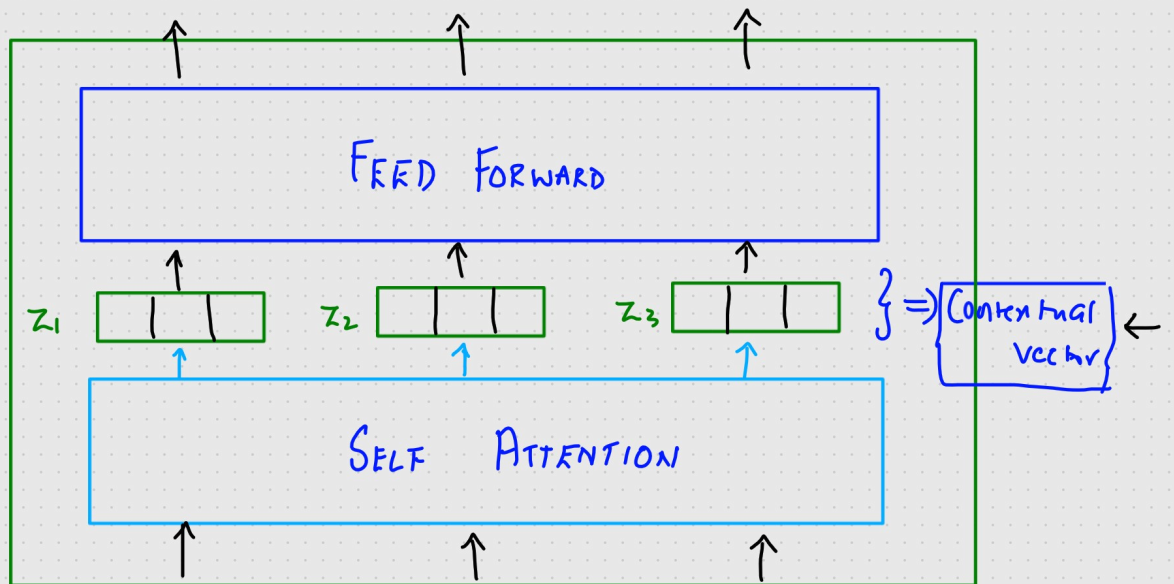


② Basic Transformer Architecture

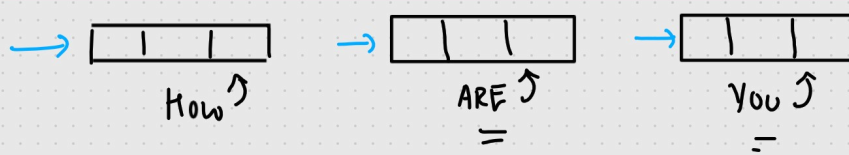
{Seq2Seq Task} → Language Translation {Eng → French}



Encoder



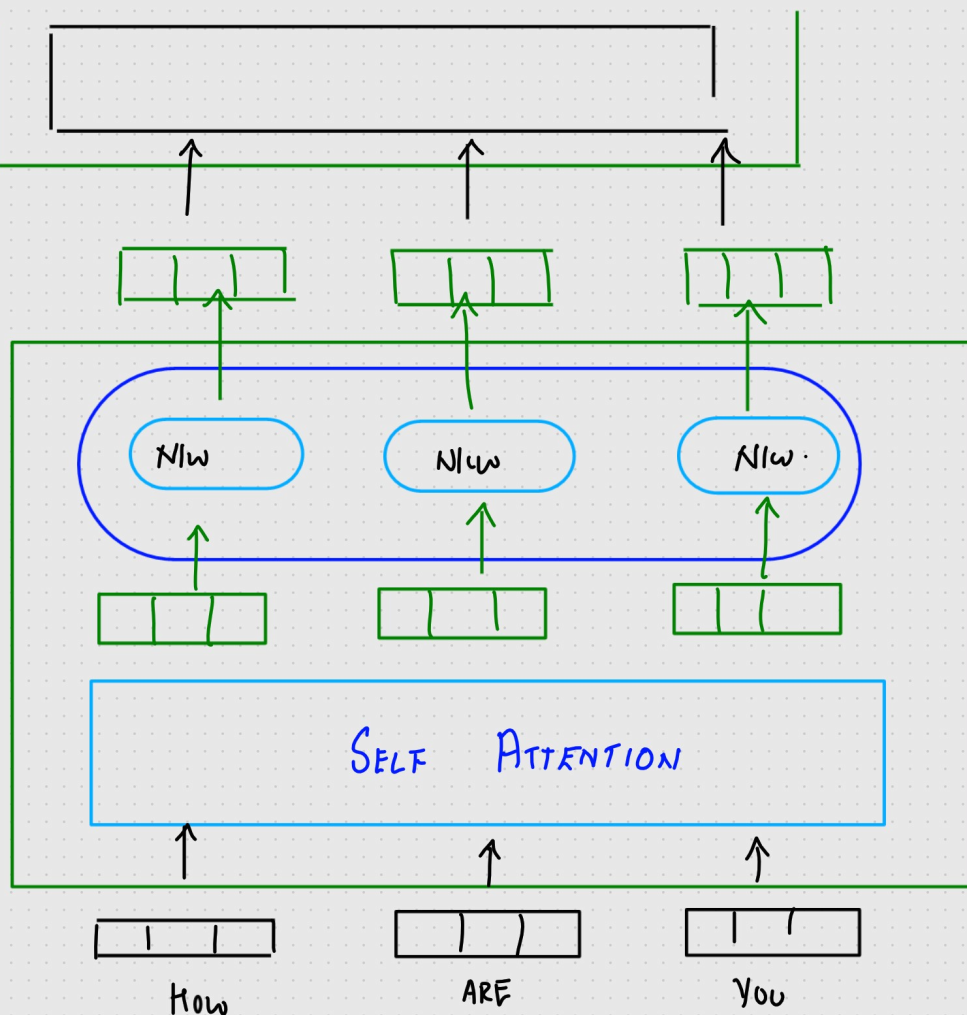
Vectors
↓
Embedding Layer



{ All the words will be passed parallelly }

Encoder 2

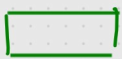
Encoder



Self Attention At a Higher Level

Eg: The cat sat on the mat, the cat lay on the rug.

Word Embedding



The

→ Cat

1 → Sat

Rank 2 → on

3 → the

mat

The

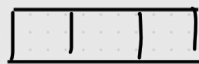
→ Cat

Sat

on

the

mat



{ Contextual Embedding }

Cat



Self Attention In Detail

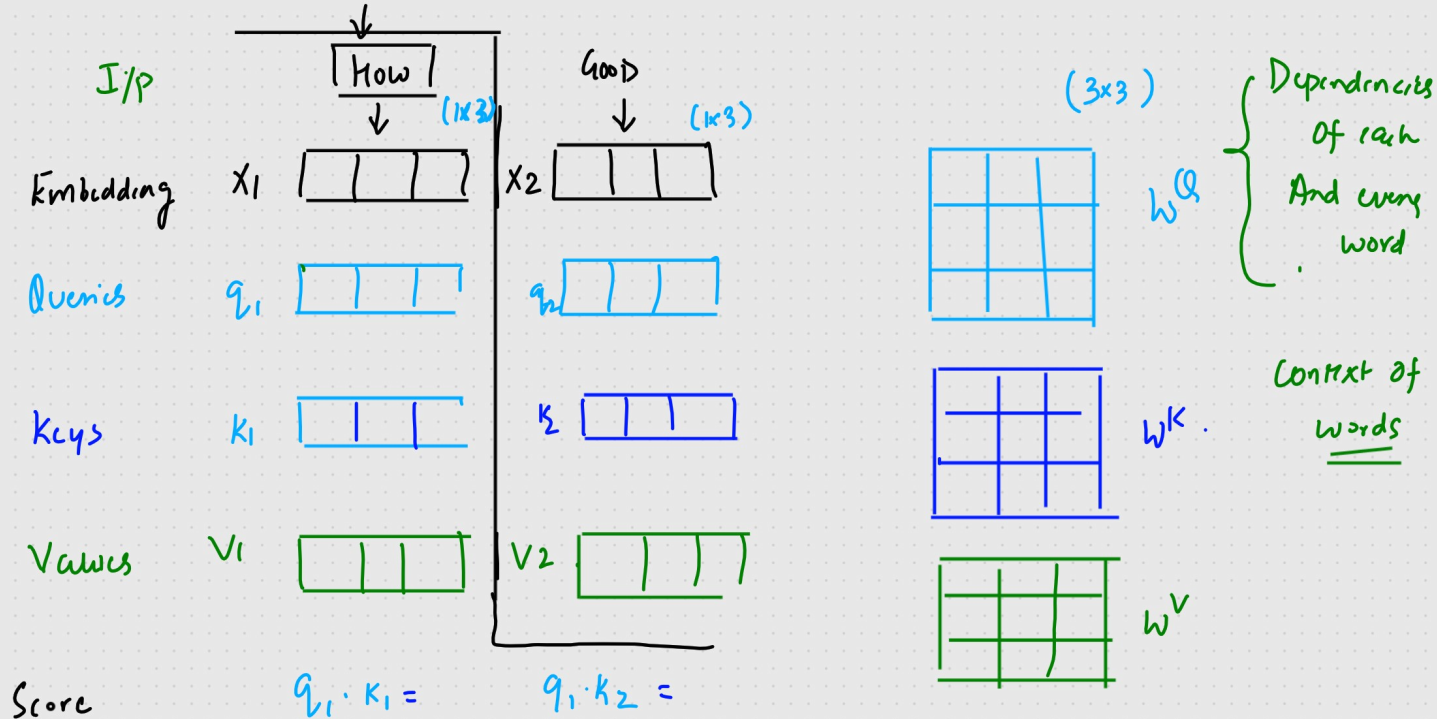
- ① To create 3 vectors from each of the encoder i/p: Query vector, Key vector, value vector. $\Rightarrow$ Contextual Embedding

Eg: YT $\Rightarrow$ Search keywords $\Rightarrow$ {Query}.

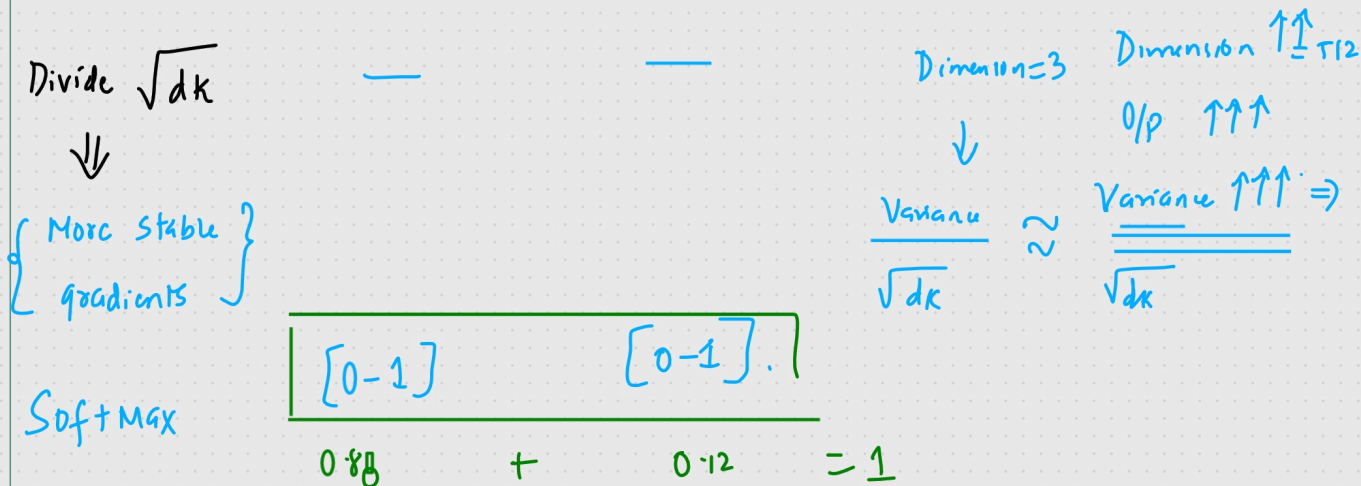
Query $\rightarrow$ {Key} $\rightarrow$ $\begin{cases} \text{Tags} \\ \text{Description} \\ \text{Title} \end{cases} \Rightarrow$ Value $\Rightarrow$ opp Video

- ② Second step in calculating self attention is to calculate the score.

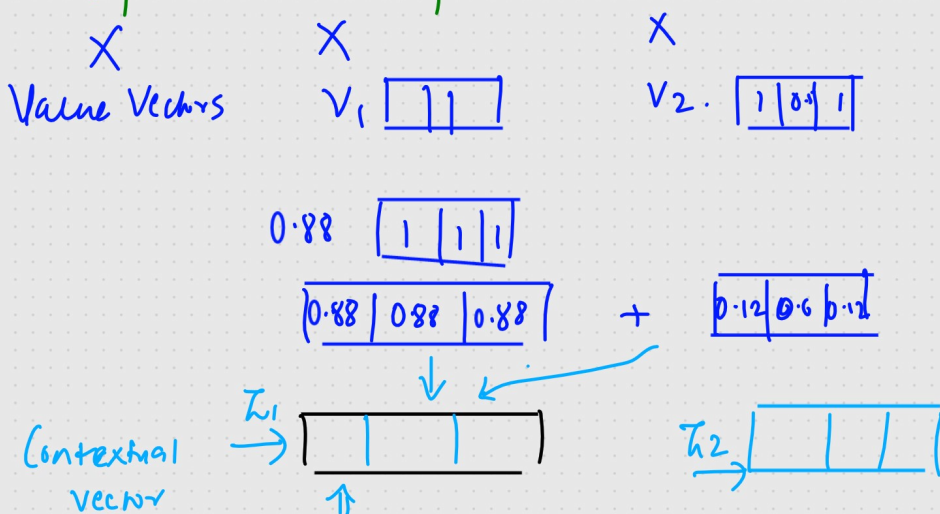
The score determines how much focus to place on the other part of the sentence.



⊛ How much Focus to place on other part of the i/p Sentence as we encode a word at certain position



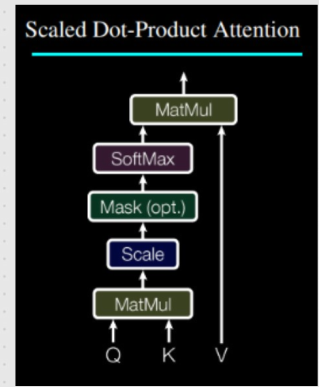
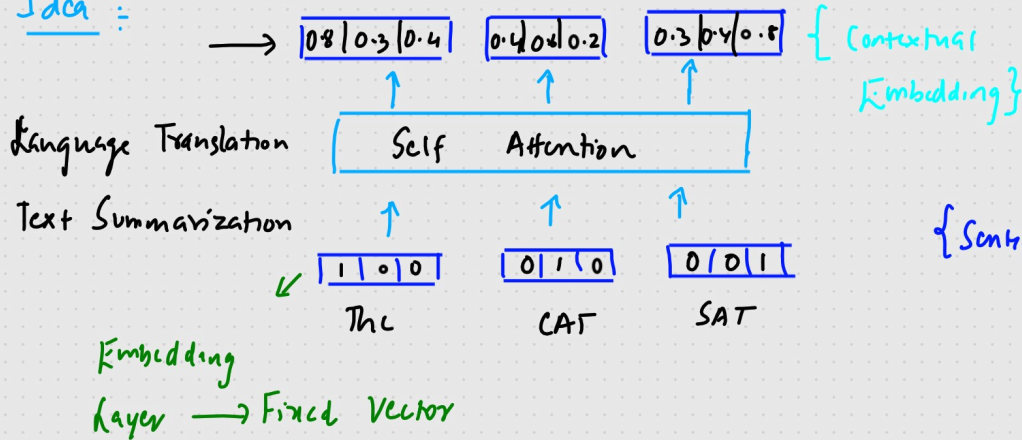
Softmax $\Rightarrow$ The Softmax score determines how much each word will be expressed at this position



Self Attention At Higher And Detailed Level

Self-attention, also known as scaled dot-product attention, is a crucial mechanism in the transformer architecture that allows the model to weigh the importance of different tokens in the input sequence relative to each other

Idea :



{ Sentence, Dataset }

1) Inputs: Queries, Keys, And Values

Model → Queries, Keys And Values

1. Query Vectors (Q):

Role: Query vectors represent the token for which we are calculating the attention. They help determine the importance of other tokens in the context of the current token.

Importance:

Focus Determination: Queries help the model decide which parts of the sequence to focus on for each specific token. By calculating the dot product between a query vector and all key vectors, the model assesses how much attention to give to each token relative to the current token.

Contextual Understanding: Queries contribute to understanding the relationship between the current token and the rest of the sequence, which is essential for capturing dependencies and context.

2. Key Vectors (K):

Role: Key vectors represent all the tokens in the sequence and are used to compare with the query vectors to calculate attention scores.

Importance:

Relevance Measurement: Keys are compared with queries to measure the relevance or compatibility of each token with the current token. This comparison helps in determining how much attention each token should receive.

Information Retrieval: Keys play a critical role in retrieving the most relevant information from the sequence by providing a basis for the attention mechanism to compute similarity scores.

3. Value Vectors (V):

Role: Value vectors hold the actual information that will be aggregated to form the output of the attention mechanism.

Importance:

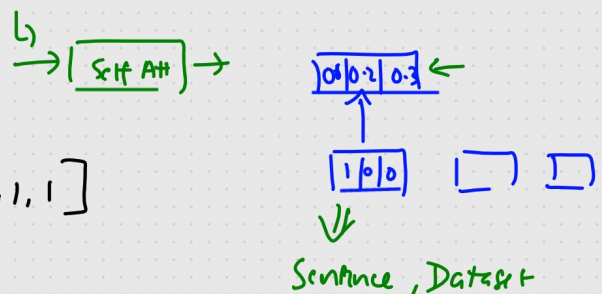
Information Aggregation: Values contain the data that will be weighted by the attention scores. The weighted sum of values forms the output of the self-attention mechanism, which is then passed on to the next layers in the network.

Context Preservation: By weighting the values according to the attention scores, the model preserves and aggregates relevant context from the entire sequence, which is crucial for tasks like translation, summarization, and more.

Input Sequence = ["The", "CAT", "SAT"]

Embedding size = 4

Q, K, V $\Rightarrow 4$



① Token Embedding

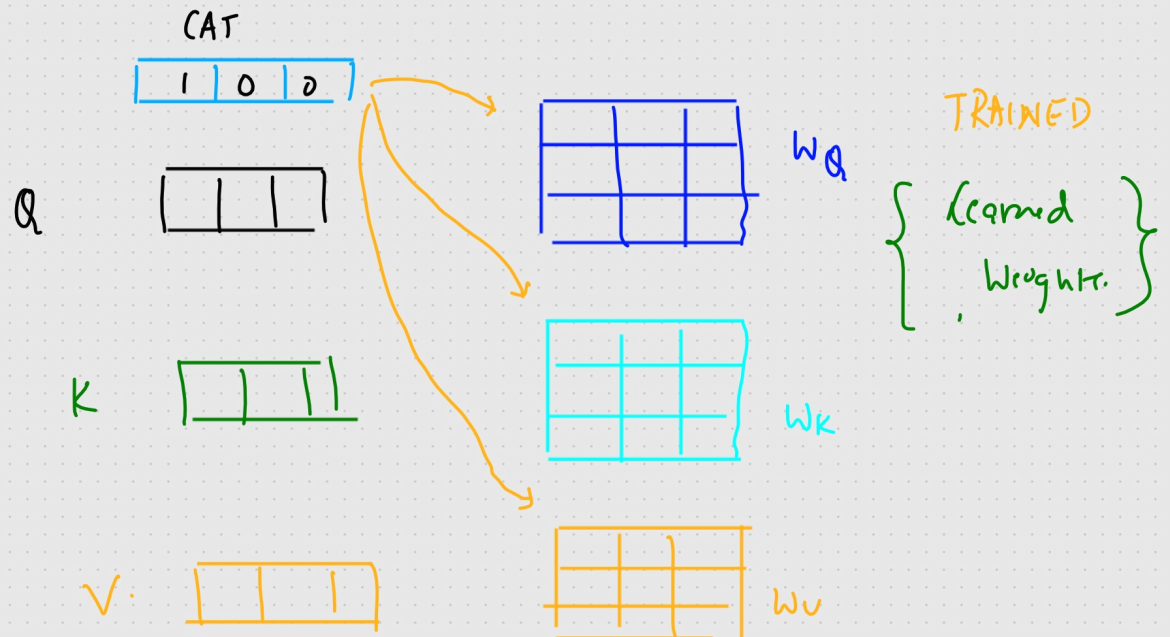
$$E_{The} = [1 \ 0 \ 1 \ 0]$$

$$E_{SAT} = [1, 1, 1, 1]$$

$$E_{CAT} = [0 \ 1 \ 0 \ 1]$$

② Linear Transformation

We create Q, K, V by multiplying the embeddings by learned weights matrices W_Q, W_K and W_V .



lets consider

$$W_Q = W_K = W_V = I$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \leftarrow$$

$$Q_{The} = \begin{bmatrix} 1 & 0 & 10 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 10 \end{bmatrix}$$

$$K_{The} = \begin{bmatrix} 1 & 0 & 10 \end{bmatrix}$$

$$V_{The} = \begin{bmatrix} 1 & 0 & 10 \end{bmatrix}$$

$$① Q_{The} = K_{The} = V_{The} = \begin{bmatrix} 1 & 0 & 10 \end{bmatrix}$$

$$② Q_{CAT} = K_{CAT} = V_{CAT} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$$

$$③ Q_{SAT} = K_{SAT} = V_{SAT} = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}$$

③ Compute Attention Scores

$$\begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

The

$$\text{Score}(Q_{\text{The}}, K_{\text{The}}) = [1 \ 0 \ 1 \ 0] \cdot [1 \ 0 \ 1 \ 0]^T = 2$$

$$\text{Score}(Q_{\text{The}}, K_{\text{CAT}}) = [1 \ 0 \ 1 \ 0] \cdot [0 \ 1 \ 0 \ 1]^T = 0$$

$$\text{Score}(Q_{\text{The}}, K_{\text{SAT}}) = [1 \ 0 \ 1 \ 0] \cdot [1 \ 1 \ 1 \ 1]^T = 2$$

For the token CAT

$$\text{Score}(Q_{\text{CAT}}, K_{\text{The}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 0 \ 1 \ 0]^T = 0$$

$$\text{Score}(Q_{\text{CAT}}, K_{\text{CAT}}) = [0 \ 1 \ 0 \ 1] \cdot [0 \ 1 \ 0 \ 1]^T = 2$$

$$\text{Score}(Q_{\text{CAT}}, K_{\text{SAT}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 1 \ 1 \ 1]^T = 2$$

For the Token SAT

$$\begin{cases} \text{Score}(Q_{\text{SAT}}, K_{\text{The}}) = [1 \ 1 \ 1 \ 1] \cdot [1 \ 0 \ 1 \ 0]^T = 2 \\ \text{Score}(Q_{\text{SAT}}, K_{\text{CAT}}) = 2 \\ \text{Score}(Q_{\text{SAT}}, K_{\text{SAT}}) = 4 \end{cases}$$

④ Scaling :

We take up the scores and scale down by dividing the scores by the

$$\sqrt{d_k} \Rightarrow d_k = 4 \quad \sqrt{d_k} = 2$$

Scaling in the attention mechanism is crucial to prevent the dot product from growing too large. $\Rightarrow$ Ensure stable gradients during Training.

d_k is large $\rightarrow$

① Gradient Exploding

② Softmax Saturation $\left\{ \begin{array}{l} \nearrow \\ \searrow \end{array} \right\} \rightarrow$ Vanishing Gradient Problem.

$$Q = \begin{bmatrix} 2 & 3 & 4 & 1 \end{bmatrix} \quad K_1 = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \quad K_2 = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}$$

Without Scaling

$$Q \cdot K_1^T = 2 \times 1 + 3 \times 0 + 4 \times 1 + 1 \times 0 = 2 + 4 = 6.$$

$$Q \cdot K_2^T = 2 \times 0 + 3 \times 1 + 4 \times 0 + 1 \times 1 = 0 + 3 + 0 + 1 = 4$$

* Score $[6, 4] \Rightarrow$ Scaling Not Applied

$$\text{Softmax}([6, 4]) = \left[\frac{e^6}{e^6 + e^4}, \frac{e^4}{e^6 + e^4} \right] = \left[\frac{e^6}{e^6(1 + e^{-2})}, \frac{e^4}{e^4(e^2 + 1)} \right]$$

① Property of Softmax

W_Q, W_K, W_V

$$([10, 1]) = [0.99, 0.01]$$

Dot product = Large values

$$= \left[\frac{1}{(1 + e^{-2})}, \frac{1}{(e^2 + 1)} \right]$$

$$\approx [0.88, 0.12]$$

Most of the attention weight is assigned to the first key vector, very little to the second vector,

With Scaling

$$\sqrt{d_k} = \sqrt{4} = \sqrt{2} \Rightarrow \text{Variance } 2, 3$$

① Compute Scaled Dot Product

$$[6, 4] \Rightarrow \text{Scale} \Rightarrow \left[\frac{6}{2}, \frac{4}{2} \right] = [3, 2]$$

dimension $\uparrow$ Variance $\uparrow 2$
 $\sqrt{d_k} \uparrow$ same

$$\text{Softmax}([3, 2]) = \left[\frac{e^3}{e^3 + e^2}, \frac{e^2}{e^3 + e^2} \right] = \left[\frac{e^3}{e^3(1 + e^{-1})}, \frac{e^2}{e^2(e + 1)} \right]$$

$$= [0.73, 0.27] \Rightarrow \text{Attention Weights}$$

* Here, the attention weights are more balanced compared to the unscaled case

Summary of Importance

Stabilizing Training: Scaling prevents extremely large dot products, which helps in stabilizing the gradients during backpropagation, making the training process more stable and efficient.

Preventing Saturation: By scaling the dot products, the softmax function produces more balanced attention weights, preventing the model from focusing too heavily on a single token and ignoring others.

Improved Learning: Balanced attention weights enable the model to learn better representations by considering multiple relevant tokens in the sequence, leading to better performance on tasks that require context understanding.

Scaling ensures that the dot products are kept within a range that allows the softmax function to operate effectively, providing a more balanced distribution of attention weights and improving the overall learning process of the model.

④ Scaling $= \sqrt{dk} = \sqrt{4} \Rightarrow 2$

$$\text{Scaled-Score} (Q_{The}, K_{The}) = \frac{2}{2} = 1$$

$$\text{Scaled-Score} (Q_{The}, K_{CAT}) = \frac{0}{2} = 0$$

$$\text{Scaled-Score} (Q_{The}, K_{SAT}) = \frac{2}{2} = 1$$

Similarly scaling
will be done for
all other Tokens.

⑤ Apply Softmax

$$\text{ATTENTION WEIGHTS}_{"The"} = \text{softmax}([1, 0, 1]) = [0.4223, 0.1554, 0.4223]$$

$$\text{ATTENTION WEIGHTS}_{"CAT"} = \text{softmax}([0, 2, 2]) = [0.1554, 0.4223, 0.4223]$$

$$\text{ATTENTION WEIGHTS}_{"SAT"} = \text{softmax}([2, 2, 4]) = [0.2119, 0.2119, 0.5762]$$

⑥ Weight Sum of Values

We multiply the attention weights by corresponding value vectors

For the Token The =

$$\text{Output}_{(\text{the})} = 0.4223 * V_{\text{The}} + 0.1554 * V_{\text{cat}} + 0.4223 * V_{\text{sat}}$$

$$= 0.4223 [1 \ 0 \ 1 \ 0] + 0.1554 [0 \ 1 \ 0 \ 1] + 0.4223 [1 \ 1 \ 1 \ 1]$$

$$= [0.4223, 0, 0.4223, 0] + [0, 0.1554, 0, 0.1554] + [0.4223, 0.4223, 0.4223, 0.4223]$$

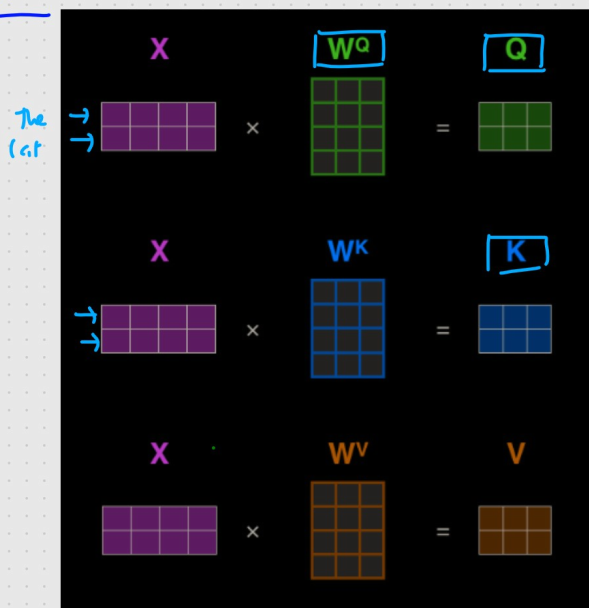
$$= [1.2669, 0.9999, 1.2669, 0.9999]$$

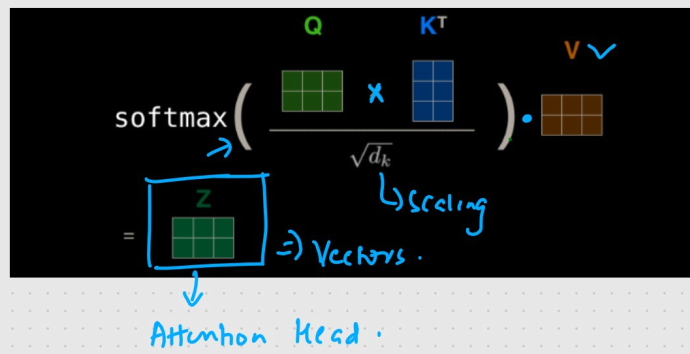
Contextual
vector

The $[1 \ 0 \ 1 \ 0] \Rightarrow \text{Self Attention} \Rightarrow [1.2669, 0.9999, 1.2669, 0.9999]$.

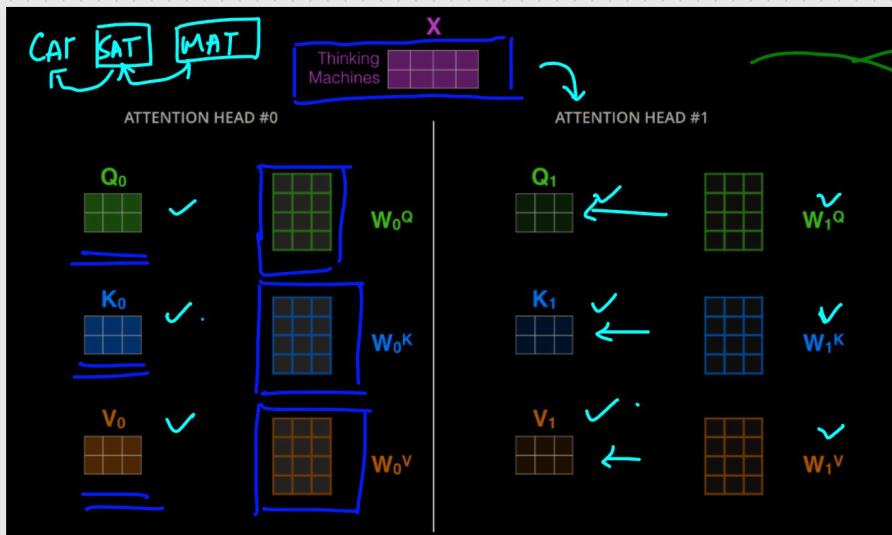
- ① $\hookrightarrow Q, K, V [W^Q, W^K, W^V]$
- ② $\hookrightarrow \text{Attention Score}$
- ③ $\hookrightarrow \text{Scaled}$
- ④ $\hookrightarrow \text{Softmax}$
- ⑤ $\hookrightarrow \text{Weighted Sum of Value (Softmax} \times V)$

④ Multi Head Attention





→ Self Attention with Multi Heads



It expands model's ability to focus on different position of tokens.

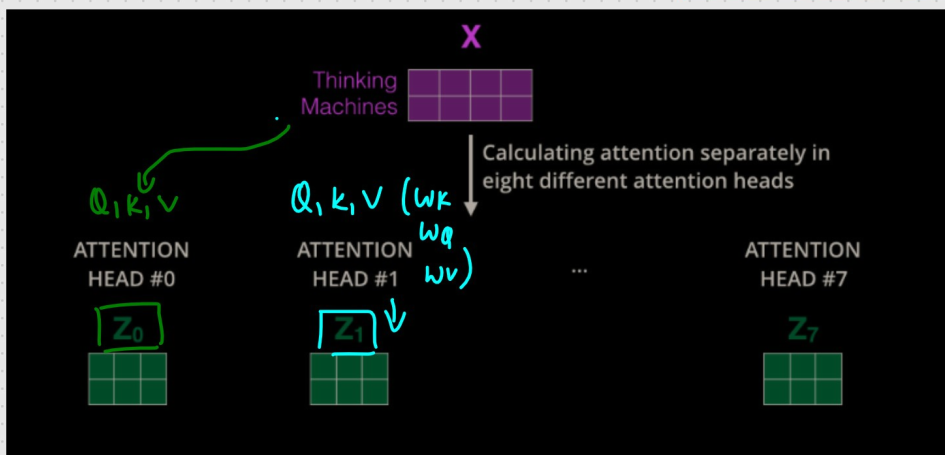
Score, Softmax $X V$

$\downarrow$
 $[\text{---}]$
 Z_0

$\downarrow$
Vectors

$\downarrow$
 $[\text{---}]$
 Z_1

Multi Head Attention



Z

Z

Z

④ Feed Forward Neural NW

